

Slay the Qt - 战斗模块 (Battle Module) 协作接入指南

大家好！本属文档旨在说明如何将外部模块（大地图、存档/读档系统、随机事件等）与战斗模块进行无缝对接。

战斗模块采用了严格的 MVC 架构与外部解耦设计。外部系统不需要直接操作任何战斗内的 UI 或底层引擎，只需要通过唯一的代理类 `BattleLauncher` 和标准数据结构 `BattleContext` 进行通信。

关键文件速览

请协作成员重点关注 `api` 文件夹下的接口文件，其余内部文件无需修改。

文件路径	核心作用	协作成员需知
<code>src/api/BattleAPI.h</code>	数据契约	定义了外部传入的数据结构 <code>BattleContext</code> 和战斗返回的结果 <code>BattleResult</code> 。
<code>src/api/BattleLauncher.h</code>	入口与出口代理	启动战斗的唯一入口，也是发送“战斗结束”信号的唯一出口。
<code>src/entities/cards/Card.h</code>	卡牌基类	存档组序列化/反序列化玩家牌组时，需参考该类的数据结构。
<code>src/entities/relics/Relic.h</code>	遗物基类	存档组序列化玩家遗物时，需参考该类的数据结构。
...		

美术资源统一放到resources文件夹下
所有代码文件请在src文件夹下搭建
每个人的代码文件放在标识性比较强的代码文件夹下以供辨别和编辑
已实例化的遗物/敌人/卡牌都在entities文件夹下，可以供调试时实例化并调用

一、如何调用战斗画面 (大地图组/事件组接入)

大地图或事件节点触发战斗时，不需要手动创建 `BattleEngine` 或 `BattleView`。请组装一份 `BattleContext` 数据，并丢给 `BattleLauncher` 启动。

1. 准备上下文数据 (BattleContext)

在进入战斗前，从你的大地图实例或当前存档数据中，提取主角状态并填入 `BattleContext`：

```
C++
#include "api/BattleLauncher.h"
#include "api/BattleAPI.h"
// 引入你需要的卡牌和遗物头文件...

void triggerBattle(const QString& enemyId) {
    BattleContext context;

    // 1. 载入玩家基础属性
    context.currentHp = 75;           // 从存档读取当前血量
    context.maxHp = 80;               // 从存档读取最大血量
    context.gold = 120;               // 从存档读取金币
    context.maxEnergy = 3;            // 当前最大能量上限

    // 2. 指定遭遇的怪物ID或种子
    context.enemySeedOrId = enemyId;

    // 3. 载入玩家当前牌组（卡牌指针的生命周期将在战斗引擎内托管）
    context.currentDeck.append(new StrikeCard());
    context.currentDeck.append(new DefendCard());
    // ... 添加更多牌

    // 4. 载入玩家当前遗物
    context.relics.append(new PenNibRelic());
    // ... 添加更多遗物

    // 5. 启动战斗！
    BattleLauncher* launcher = new BattleLauncher(this);

    // 建立信号监听（见下文“二、如何衔接存档与地图”）
    // connect(launcher, ...);

    launcher->launch(context);
}
```

⚠ 内存管理注意：

放入 `BattleContext` 的 `Card*` 和 `Relic*`，其控制权将移交给战斗引擎。战斗模块内部会妥善管理它们的生命周期。如果在外部依然保留了这些指针，请注意不要在外提前 `delete` 它们。

📁 二、战斗结算与存档、地图的衔接

当战斗自然结束（无论是胜利、死亡，还是逃跑），`BattleLauncher` 都会发射一个带数据的 `battleConcluded` 信号。地图组和存档组必须监听这个信号，以刷新大地图状态或执行自动存档。

1. 监听战斗结束信号

在调用 `launch()` 之前，将大地图/存档管理器的槽函数连接到发射器上：

C++

```
BattleLauncher* launcher = new BattleLauncher(this);

// 监听战斗结束的回调
connect(launcher, &BattleLauncher::battleConcluded,
        this, [this, launcher](BattleResult result) {

    // 1. 判断生死，决定游戏走向
    if (!result.isVictory) {
        // 主角阵亡，触发 GameOver 界面或删档逻辑
        showGameOverScreen();
        launcher->deleteLater();
        return;
    }

    // 2. 战斗胜利，同步战斗损耗与战利品到全局存档
    GlobalSaveData::updatePlayerHp(result.remainingHp);
    GlobalSaveData::updatePlayerGold(result.currentGold);

    // 3. 获取战斗结束后的新卡牌或遗物状态（例如：损耗的遗物层数）
    GlobalSaveData::syncRelics(result.battledRelics);
    GlobalSaveData::syncDeck(result.finalDeck);

    // 4. 刷新大地图 UI，解锁下一层节点
    MapManager::unlockNextNode();

    // 5. 安全销毁战斗代理
    launcher->deleteLater();
});

launcher->launch(context);
```

📁 三、给协作成员的开发约定

- 1 绝对隔离原则：地图组和存档组请不要尝试直接 `#include "BattleEngine.h"` 或 `#include "BattleView.h"`。任何需要传递给战斗引擎的数据，或者需要从战斗引擎拿出来的数据，全部在 `BattleAPI.h` 的结构体中增删字段。
- 2 场景切换规范：`BattleLauncher` 在内部使用 `QGraphicsScene` 或 `QWidget` 切换机制。大地图模块在接收到 `battleConcluded` 信号时，应当重新将自己的 `Scene` 或 `Widget` 设为应用的中央视图（`Central Widget`）。
- 3 数据深浅拷贝：传入 `BattleContext` 的牌库列表将被视为本次战斗的专属副本，引擎可能会在战斗中对其进行诸如“升级”、“消耗”、“转化”等操作。返回的 `BattleResult` 中包含的是战斗结束时的牌组状态，存档组请以返回的牌组覆盖原有存档数据。

如有任何接口扩展需求（例如需要传入药水系统、自定义背景图片等），请随时与战斗模块负责人沟通更新 `BattleAPI.h` 喵！

4. 初次编译与轰鸣起飞

- 建议先进行一次彻底的清理：点击左下角的“锤子”图标上方的项目图标，或者点击菜单栏 `构建 (Build)` -> `重新构建项目 (Rebuild Project)`。
- 清理完毕后，直接按下神级组合键：`Ctrl + R` (运行)。
- 如果看到大史莱姆战场华丽弹出，说明本地环境完全跑通！

协作成员编辑指引

环境跑通后，大家就可以根据分工各司其职了：

-  负责大地图与事件的同学：
 - 请先仔细阅读根目录下的 `README_BattleModule.md`。
 - 你们的开发主战场在你们自己的地图模块。当需要切入战斗时，请参考文档，通过 `BattleLauncher` 组装 `BattleContext` 即可调出画面。
-  负责存档与读档的同学：
 - 你们需要关注 `src/api/BattleAPI.h` 中的 `BattleResult` 结构。
 - 监听 `BattleLauncher::battleConcluded` 信号，在 Lambda 表达式中拿到战斗结束后的血量、金币、剩余牌组和遗物层数，并写入你们的全局存档系统。
-  负责新卡牌与新遗物设计的同学：
 - 做新卡：在 `src/entities/cards/` 文件夹下，参考现有的卡牌（如 `BurningPactCard.h`），继承 `Card` 基类并重写 `play()` 逻辑。
 - 做新遗物：在 `src/entities/relics/` 文件夹下，参考 `PenNibRelic.h`（钢笔尖），继承 `Relic` 基类并重写你们需要的战斗钩子（Hooks）。

Git 协作小贴士（如果使用 Git 的话）

我们在提交代码时，绝对不要把各自电脑上自动生成的 `build-SlayTheQt-xxx` 文件夹和 `SlayTheQt.pro.user` 文件提交到版本库中，记得把它们加进 `.gitignore` 喵！

祝大家开发愉快，让我们一起把这个尖塔卡牌游戏做完吧！🚀💣🍷

Slay the Qt 开发组：一分钟快速开工指南

各位代码大佬！拿到纯净版的工程压缩包后，请按照以下步骤在各自的电脑上配置并启动项目：

1. 解压项目（避坑第一步）

- 将压缩包解压到本地。
-  铁律：解压的绝对路径中千万不能包含中文或空格（例如 `D:\我的作业\Qt项目\` 是绝对不行的，会导致 qmake 或 MinGW 编译器莫名其妙报错）。建议存放在类似 `D:\DevProjects\SlayTheQt\` 这样的纯英文路径下。

2. 用 Qt Creator 打开项目

- 打开你的 Qt Creator。
- 点击左上角 `文件 (File)` -> `打开文件或项目 (Open File or Project)`。
- 浏览到刚解压的文件夹，选中 `SlayTheQt.pro` 核心工程文件，点击打开。

3. 激活项目：配置 Kit (Configure Project)

- 因为工程里没有旧的本地配置文件，Qt Creator 打开后会弹出一个充满复选框的 "Configure Project"（配置项目）界面。
- 操作：在列表中勾选你本地安装的编译器（例如 `Desktop Qt 5.14.2 MinGW 64-bit` 或更高版本）。
- 点击：点击右下角大大的 `Configure Project` 按钮。此时 Qt Creator 会在左侧的项目树中完美亮起所有源文件！